Matière : Développement Mobile

Auditoire : GLSI-3

ISTIC
Higher Institute of Information and Communication Technologies
المعهد العالي لتكنولوجيات المعلومات والاتصال

TP2 : Stateless Widgets en Flutter

Activité 1 : MaterialApp

1. Créer un nouveau projet Flutter que vous nommez « **tp2**»
2. Accéder au fichier « lib/**main.dart** » et effacer tout son contenu et le remplacer par le code ci-après :

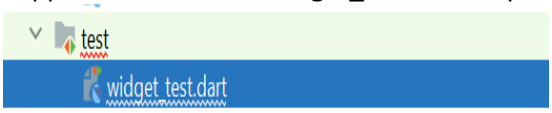
```
void main() {
  runApp(
    MaterialApp()
  );
}
```

NB

- La fonction **main** lance l'exécution de votre application en appelant la fonction **runApp**
- runApp(….) prend en argument un objet Correspondant au Widget à créer (il peut être un Widget prédéfinit ou un widget que vous créez). Etant donnée que notre interface mobile suit le design Materila, nous allons donc encapsuler nos interfaces à l'intérieur du Widget MaterialApp.

3. Exécuter le code obtenu, remarquez que vous obtenez une fenêtre noire. Vous devez alors configurer **MaterialApp** afin d'avoir l'interface que vous souhaiterez.

Supprimer le fichier « **widget_test.dart** » qui se trouve sous le dossier « **test** »



4. Nous allons maintenant paramétrer **MaterialApp**. Notre interface mobile comprend un simple texte « Ex1 Flutter ». Il suffit de remplacer **MaterialApp()** par le code suivant. Exécuter

```
MaterialApp(
debugShowCheckedModeBanner: false,   //désactiver étiquette de debug
title: 'EX1 TP2 Flutter',           //titre de l'application
home: //contiendra la première interface qui sera lancée
        Text('EX1 Flutter') //Afficher un texte
    )
```

5. Afin de modifier le thème des couleurs de votre application, veuillez ajouter la propriété **theme** à votre MaterialApp comme dans cet exemple :

```
        theme: ThemeData(useMaterial3: false, primarySwatch: Colors.amber)
```

ThemeData est utilisé pour configurer l'apparence de l'ensemble de l'application

Activité 2 : Scaffold

Afin d'améliorer notre affichage, nous allons intégrer notre texte au sein du widget **Scaffold**.

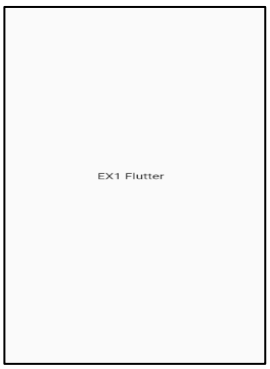
Scaffold *: est un Widget issu de la bibliothèque Material, fournit une barre d'application par défaut, un titre et un corps qui contient l'arborescence de widgets de l'écran d'accueil*
Center *: un Widget permet de centrer son Widget fils*

1. Modifier l'attribut **home** du Widget MaterialApp afin de faire apparaître un Scaffold.

Code

```
home:
      Scaffold(
                body: Center(child:Text('EX1 Flutter'))
              )
```

Affichage



2. Ajouter un peu de style à votre texte : remplacer alors `Text('EX1 Flutter')` par :

```
    Text('EX1 Flutter',
          style: TextStyle(
                                fontSize: 30,
                                fontWeight: FontWeight.bold)
                                )
)
```

3. Nous allons maintenant configurer la barre de l'application à travers le widget Scaffold afin d'obtenir l'apparence suivante :

Vous devez alors ajouter l'attribut **appBar** à **Scaffold** et le paramétrer comme demandé :

```
Scaffold(
    appBar: AppBar(
        title: Text('Barre'),
        leading: Icon(Icons.arrow_back_sharp),
        actions: [
          Icon(Icons.add_shopping_cart),
          Icon(Icons.email),
          Icon(Icons.phone)
        ]),
    body: Center(
        child: Text(
      'EX1 Flutter',
      style: TextStyle(fontSize: 30, fontWeight: FontWeight.bold),
    )) //Afficher un texte,
    )
```

6. Ajouter de plus la propriété **floatingActionButton** au Scaffold afin d'afficher l'icone en bas de l'écran

```
floatingActionButton: FloatingActionButton(child:Icon(Icons.home),
                      onPressed: (){
                    //create a snack bar
                    SnackBar sn = const SnackBar(content: Text("Vous m'avez tappé"));
                    ScaffoldMessenger.of(context).showSnackBar(sn);
})
```

Activité 3 : Image

À la place du texte, nous allons afficher une image locale au projet.

1. Créer un nouveau dossier dans votre projet puis l'appeler « images »
2. Y ajouter une image de votre choix, par exemple l'image «**pokemon.jpg**». Ouvrir le fichier « **pubspec.yaml** » et chercher le bout de code contenant « assets » et qui est comme suit :

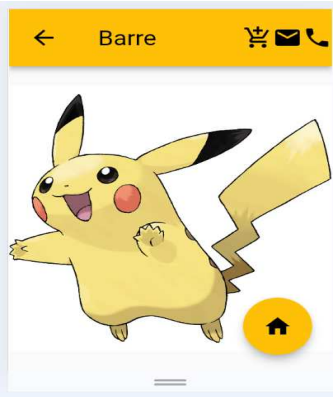
```
# assets:
#   - images/a_dot_burr.jpeg
#   - images/a_dot_ham.jpeg
```

3. Remplacer ce dernier afin de mentionner le chemin vers l'image correspondante (ou le dossier image) puis cliquez sur « **Pub get** » ou bien lancer dans le terminal « **flutter pub get** » pour récupérer les dépendances du fichier « **pubspec.yaml** » et synchroniser l'application.

4. Modifier le body du Scaffold afin de faire apparaître notre image tout en mentionnant son chemin (***images/pokemon.jpg***). Voici le code de maj du body du Scaffold

```
body:Center(child:  Image.asset('images/pokemon.jpg'))
```

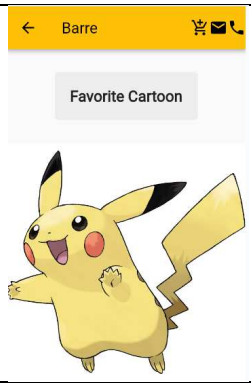
Vous devez alors obtenir le résultat suivant :



Activité 4 : Widget Column

Maintenant nous voulons afficher du texte « **Favorite Cartoon** » avant l'image. Pour atteindre cet objectif nous devons utiliser le Widget Column qui acceptera plusieurs Widgets à positionner verticalement.

Code	Résultat

`body:`
 	
`Column(`
`children:[`
 	
`Text('FavoriteCartoon'),`
 	
`Image.asset("images/pokemon.jpg")`
 	
`]`
`)`
	

Pour ajouter un peu de style au text Favorite Cartoons (padding, margin, …), utilisez le widget **Container** et *y inclure le widget Text à styler.*

Text("Favorite Cartoon")

Remplacé par

```
Container(
    child: Text('Favorite Cartoons',
                style:
                TextStyle(fontSize: 20, fontWeight: FontWeight.bold)),
    padding: EdgeInsets.all(20),
    margin: EdgeInsets.all(30),
    decoration: new BoxDecoration(
                                color: Colors.grey.shade200,
                                borderRadius: BorderRadius.all(
                                Radius.circular(5))
                               )
), //fin Container
```

Pour afficher la colonne Centré, nous allons l'intégrer dans un Widget Center.

Structure générale du code
`body:` `Center(child:Column(children: [….]  ))`

Notre code devient de plus en plus complexe. Nous allons avoir recours donc à le séparer dans divers fichiers. Le fichier main appelle le Widget à exécuter. Un autre fichier contenant la structure du Widget.

1) Créer sous le dossier « **lib** » un nouveau fichier **dart** que vous appelez « **FirstUI** ».
2) Créer la classe « **FirstUI**» et déplacer le code permettant de créer votre interface à partir du Scaffold dans ce fichier. **Cette classe doit hérité de StatelessWidget et doit redéfinir la fonction build**

Le fichier main.dart devient alors de cette forme :

```
import 'FirstUI.dart';

void main() {
  runApp(MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'EX1 TP2 Flutter',
      theme: ThemeData(useMaterial3: false, primarySwatch: Colors.amber),
      home: FirstUI()));  //du le la classe relative à l'interface à crééer
}
```

Le fichier FirstUI.dart contient le code suivant :

```
import 'package:flutter/material.dart';

class FirstUI extends StatelessWidget {


  @override
  Widget build(BuildContext context) {
    return Scaffold(
            appBar: AppBar(actions: [
                            Icon(Icons.add_shopping_cart),
                            Icon(Icons.email),
                            Icon(Icons.phone)],
                            title: Text('Barre'),
                            leading: Icon(Icons.arrow_back_sharp)),
                            floatingActionButton: FloatingActionButton(
                             onPressed: () {
              //create a snack bar
              SnackBar sn = const SnackBar(content: Text("Vous m'avez tappé"));
              ScaffoldMessenger.of(context).showSnackBar(sn);
                                        },
            child: Icon(Icons.home)),
            body: Column(
            children: [
                        Container(
                                    child: Text("Favorite Cartoon",
                                           style: TextStyle(fontSize: 20,
fontWeight: FontWeight.bold)),
              padding: EdgeInsets.all(20),
              margin: EdgeInsets.all(30),
              decoration: BoxDecoration(
                  color: Colors.grey.shade200,
```

Cette fonction **retourne** la structure de l'interface mobile à créer

Vous devez y implémenter la structure de votre interface

```
                    borderRadius: BorderRadius.all(Radius.circular(5))),
            ),
            Image.asset("images/pokemon.bmp")
          ],
        ));
  }
}
```

Activité 5 : Widget Row

On vous demande maintenant d'ajouter après l'image de notre interface une **ligne** (Row) contenant deux boutons « **Sign Up** » et « **Sign In** ». Vous êtes alors besoins d'utiliser le Widget **Row**. Ce dernier doit être intégré comme un nouveau Fils dans la **Column** déjà créée. Voici la *structure générale* du nouveau code :

```
body: Center( child:Column(children: [
  Container(
       ….),

  Image.asset("images/pokemon.jpg")

,
  Row(
     //diviser l'espace vide dans la ligne entre les boutons
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,

      children: [
        //code pour le premier bouton
        ElevatedButton(onPressed:() {}, child: Text('Sign Up')),
        //code pour le deuxième bouton
        ElevatedButton(onPressed:() {}, child: Text('Sign In'))
      ]
  )

]))
```

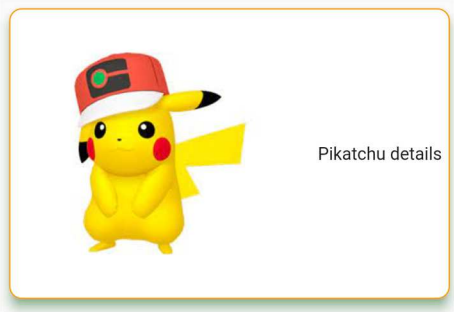
Ajouter aux fonctions oOpressed le code nécessaire pour afficher un SnackBar indiquant le bouton cliqué.

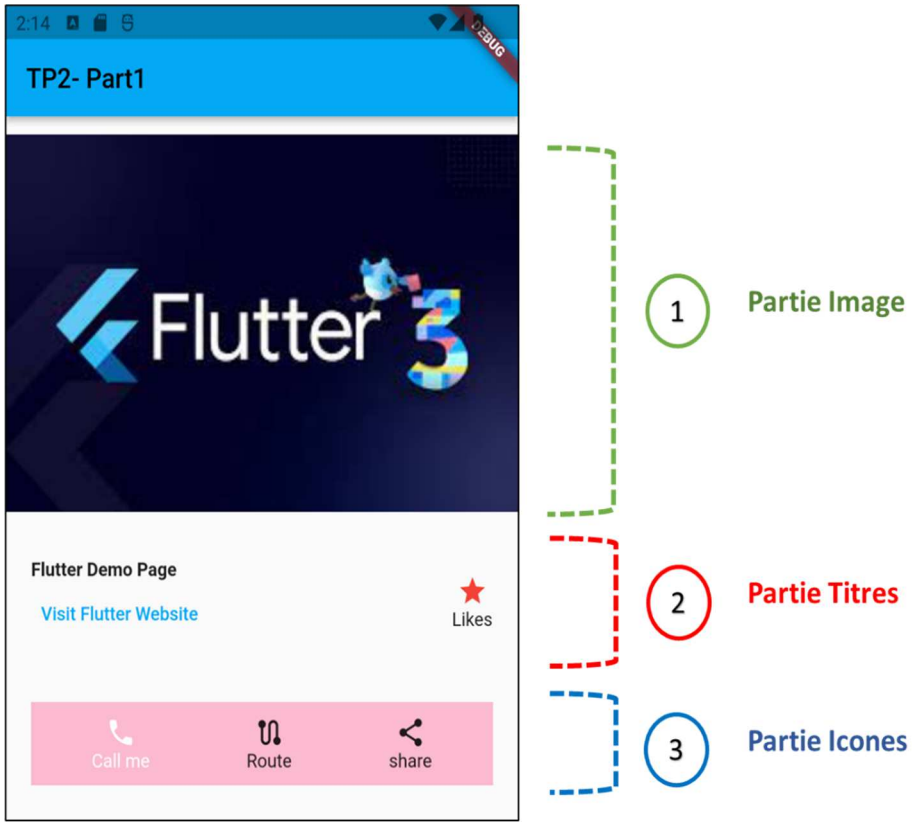
Activité 6 : Widget Card

Card : c'est un Widget utilisé pour représenter toutes les informations similaires dans un seul bloc. Lors de la conception d'une carte, nous pouvons y ajouter des éléments tels que du texte, des images, des vidéos, des boutons de texte, etc.

Le lien [https://codesinsider.com/flutter-card-example-tutorial/] pourra vous aider.

Travail : Créer un nouveau fichier dart « SecondUI » et y inclure le code nécessaire pour créer une simple Carte comme suit.



Exercice

Créer l'interface suivante en la décomposant en 3 parties comme indiqué dans la figure